

MA3632 Lecture 3 – Data Transformation

Lectures 1 and 2 addressed the opening stages of the data mining pipeline: acquiring data and cleaning it of errors, missing values, and structural defects. The data emerging from those stages is correct but not necessarily in a form that algorithms can use effectively. This lecture covers *data transformation*: the family of operations that convert cleaned data into a representation better suited to statistical modelling.

Four categories of transformation are considered. Scaling and normalisation bring numerical variables onto comparable scales. Categorical encoding converts non-numerical variables into numerical representations without misrepresenting their structure. Distribution transformations address asymmetry and heavy tails in numerical features. The lecture closes with a discussion of *data leakage* – the most consequential mistake that arises when transformations are applied carelessly relative to the train/test split.

1 Scaling and Normalisation

Many learning algorithms are sensitive to the absolute magnitudes of numerical features. A dataset containing age (range roughly 0–100) and annual income (range roughly 10^4 – 10^6) will cause gradient-based optimisers and distance-based methods to weight income far more heavily than age, simply because its values are larger. Scaling addresses this by mapping each feature to a common range or to a distribution with fixed moments.

1.1 Min-max scaling

Definition 1.1. The **min-max scaler** maps each observation x_i of a variable X to

$$\tilde{x}_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}},$$

where $x_{\min} = \min_i x_i$ and $x_{\max} = \max_i x_i$. The transformed values lie in $[0, 1]$. More generally, one can scale to an arbitrary interval $[a, b]$ by setting $\tilde{x}_i = a + (b - a)(x_i - x_{\min})/(x_{\max} - x_{\min})$.

Min-max scaling preserves the shape of the distribution exactly: it is an affine transformation, so relative spacings between observations are unchanged. Its weakness is sensitivity to outliers. If a single observation takes an extreme value, the denominator $x_{\max} - x_{\min}$ is dominated by that value, and the remaining observations are compressed into a small subinterval of $[0, 1]$.

1.2 Standardisation

Definition 1.2. The **standard scaler** (or *z-score normalisation*) transforms each observation to

$$\tilde{x}_i = \frac{x_i - \bar{x}}{s},$$

where $\bar{x}$ and s are the sample mean and standard deviation computed on the training data. The transformed variable has sample mean zero and sample standard deviation one.

Standardisation is useful when features measured on different scales need to be made comparable, particularly for distance-based methods and algorithms involving regularisation or gradient-based optimisation. It does not make a non-Gaussian variable Gaussian, and remains sensitive to outliers: it does not bound the transformed values to any fixed interval, so a very large observation will still produce a very large $\tilde{x}_i$.

1.3 Robust scaling

Definition 1.3. The **robust scaler** replaces the mean and standard deviation with robust measures of location and spread:

$$\tilde{x}_i = \frac{x_i - \text{median}(X)}{Q_3 - Q_1},$$

where Q_1 and Q_3 are the first and third quartiles of the training data.

Because the median and IQR are resistant to extreme values, the robust scaler is preferable when outliers survive the cleaning stage or when the distribution of X is substantially skewed. The transformed variable does not generally have mean zero or unit variance, but its bulk is brought onto a comparable scale regardless of the tails.

Remark 1.4. All three scalers share an important structural property: the parameters ($x_{\min}$, $x_{\max}$; $\bar{x}$, s ; median, IQR) must be estimated on the *training data only* and then applied to the test data using those training estimates. Estimating parameters on the full dataset before splitting leaks information about the test set into the transformation, invalidating the test evaluation. This point is treated formally in Section 4.

Example 1.5. A dataset has two features: $X_1 \in \{25, 30, 35, 40, 45\}$ (age) and $X_2 \in \{30000, 45000, 60000, 75000, 200000\}$ (income, with one outlier). After min-max scaling, the outlier in X_2 maps to 1 and the remaining values are compressed into $[0, 0.265]$. After robust scaling (using the median 60000 and IQR 30000), the outlier maps to $(200000 - 60000)/30000 \approx 4.67$, which is large but does not compress the other values. Standardisation gives the outlier a z -score of approximately 1.73. In this case the robust scaler best preserves the structure of the non-outlying observations.

1.4 Quantile transformation

The three scalers above are all affine: each is of the form $\tilde{x}_i = a + b x_i$ for constants a, b determined by the training data, so none of them can change the *shape* of a variable's distribution, only its location and spread. When altering the marginal shape is desirable – for example, to reduce extreme skew or heavy tails – a nonlinear transformation such as a quantile transformation may be considered.

Definition 1.6. Let $\hat{F}$ denote the empirical cumulative distribution function of X computed on the training data. The **quantile transformation** maps each observation to its empirical rank,

$$\tilde{x}_i = \hat{F}(x_i) \in [0, 1].$$

Composing with the standard normal quantile function Φ^{-1} gives the **Gaussian quantile transform**, $\tilde{x}_i = \Phi^{-1}(\hat{F}(x_i))$, whose output is approximately standard normal by construction.

Remark 1.7. Because $\hat{F}$ is generally nonlinear, the quantile transform does not preserve relative spacing: two observations that are close together in a dense region of the distribution can end up far apart after transformation, and vice versa in a sparse region. This is a deliberate trade: the transform forces a target marginal shape at the cost of distorting distances. It can be useful when controlling marginal distributions is beneficial, for example to reduce the influence of extreme feature values or to make certain modelling relationships easier to represent, but should be used cautiously with distance-based methods, since it changes relative distances between observations. It has essentially no effect on most tree-based methods, since splits depend only on the ordering of values, and is generally unnecessary in that setting. As with the affine scalers, $\hat{F}$ must be estimated on the training data only and then applied, unchanged, to the test data; refitting it on the test set is exactly the leakage problem discussed in Section 4.

2 Encoding Categorical Variables

Categorical variables must be converted to numerical form before most learning algorithms can process them. The appropriate encoding depends on the measurement scale of the variable (nominal or ordinal) and on the number of categories.

2.1 One-hot encoding for nominal variables

A nominal variable C taking k distinct values $c_1, \dots, c_k$ cannot be encoded as the integers $1, \dots, k$ because this would impose an ordering ($c_1 < c_2 < \dots < c_k$) that does not exist. The standard remedy is one-hot encoding.

Definition 2.1. The **one-hot encoding** of a nominal variable C with categories $c_1, \dots, c_k$ replaces C with k binary indicator variables $I_1, \dots, I_k$, where $I_j = 1$ if $C = c_j$ and $I_j = 0$ otherwise.

If all k indicators are retained, the columns $I_1, \dots, I_k$ satisfy the linear constraint $\sum_{j=1}^k I_j = 1$ for every observation. In a regression model with an intercept term, this introduces perfect multicollinearity: the design matrix is rank-deficient and the coefficient vector is not identified. The standard fix is to drop one indicator and use only $k - 1$ columns; the dropped category then serves as the reference level absorbed into the intercept.

Proposition 2.2. Let $\mathbf{X}$ be a design matrix for a linear model with intercept, and suppose a nominal variable with k categories is represented by all k indicator columns. Then $\text{rank}(\mathbf{X}) < p$, where p is the total number of columns, and the ordinary least squares estimator $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ does not exist.

Proof. The column of ones (intercept) equals the sum of the k indicator columns, so the columns are linearly dependent. Hence $\mathbf{X}^\top \mathbf{X}$ is singular. $\square$

Remark 2.3. The dummy variable trap does not arise in tree-based models (decision trees, random forests, gradient boosting), which do not form a design matrix or require matrix inversion. In those settings, dropping a category is harmless but unnecessary.

2.2 Ordinal encoding

For an ordinal variable the categories have a natural ordering, and this ordering should be reflected in the encoding. One-hot encoding discards the ordering information entirely. Integer encoding preserves the ordering but assigns equally spaced numerical codes; models that interpret those numerical differences quantitatively may therefore impose an equal-spacing assumption that is not substantively justified.

Definition 2.4. An **ordinal encoding** assigns integer values $0, 1, \dots, k - 1$ to the categories of an ordinal variable in accordance with their natural order.

The assumption of equal spacing is often an approximation. The difference in achievement between a grade of “fail” and “pass” is not necessarily the same as the difference between “pass” and “distinction”, and treating them as equal introduces a form of measurement error. In practice, ordinal encoding is nevertheless widely used for ordered categories, with the understanding that it is an approximation.

2.3 High-cardinality categoricals and target encoding

When a nominal variable has many categories – postcodes, product identifiers, user IDs – one-hot encoding produces a very large number of columns, most of which will be zero for any given observation (a *sparse* representation). This is computationally wasteful and can degrade model performance.

Definition 2.5. Target encoding replaces each category c_j of a nominal variable with the conditional mean of the target variable among training observations in that category:

$$\hat{\mu}_j = \frac{1}{|\{i : C_i = c_j\}|} \sum_{\{i : C_i = c_j\}} y_i.$$

Target encoding is compact (one column regardless of the number of categories) and can capture the relationship between the categorical variable and the target. Its principal danger is *in-sample target leakage* (or overfitting): when $\hat{\mu}_j$ is computed using the full training set, the encoded value for each training observation i is constructed using y_i itself, so the feature encodes information about the very target it is meant to predict. The fitted encoding can still be applied, once constructed, to transform genuinely unseen test observations; the problem is specific to how each training observation’s own feature value is built from its own target. This is addressed by computing the target mean using all training observations *except* i (leave-one-out encoding) or by using cross-validation folds.

2.4 Frequency encoding and the hashing trick

Target encoding is not always available or desirable: it requires a target variable to exist (ruling it out for unsupervised tasks) and, even with leave-one-out correction, can be unstable and prone to overfitting for categories with very few observations. Two alternatives avoid the target entirely.

Definition 2.6. Frequency encoding replaces each category c_j with its empirical frequency (or raw count) among the training observations,

$$\hat{f}_j = \frac{|\{i : C_i = c_j\}|}{n_{\text{train}}}.$$

Frequency encoding uses no information about y , so it carries no target-leakage risk at all, at the cost of losing any distinction between two categories that happen to occur equally often but have different relationships with the target.

Definition 2.7. Given a fixed number of buckets m , the **hashing trick** maps each category c_j to the column index $h(c_j) \bmod m$ for a hash function h , and increments (or sets) that column rather than allocating a dedicated indicator per category.

Remark 2.8. The hashing trick trades exactness for a fixed, bounded number of columns: distinct categories that collide under the hash function share a column and become indistinguishable to the model. It is used when even one-hot or target encoding is impractical because the number of categories is unbounded or unknown in advance (user identifiers in a streaming system, for example), and m is chosen large enough that the collision rate is tolerable for the application.

3 Transforming Skewed Distributions

Transformations of skewed variables can be useful when they make relationships more nearly linear, stabilise variance, or reduce the influence of extreme values. Right-skewed variables arise naturally in applications such as income, property prices, website traffic, and biological measurements, which frequently follow approximately log-normal or power-law distributions.

3.1 The logarithmic transformation

If $X > 0$ and is right-skewed, the transformation $Y = \log X$ often produces an approximately symmetric distribution. This is because, if X is log-normally distributed – that is, $\log X \sim N(\mu, \sigma^2)$ – then Y is exactly normal. More generally, the log transformation compresses the right tail and stretches the left tail, reducing skewness.

Remark 3.1. If X takes the value zero, $\log X$ is undefined. The common remedy is to use $\log(X+1)$ or, more generally, $\log(X+c)$ for a small constant $c > 0$. The choice of c is somewhat arbitrary and can affect the degree of skewness reduction.

3.2 The Box-Cox family

The log transformation is a special case of a one-parameter family due to Box and Cox (1964).

Definition 3.2. For $X > 0$, the **Box-Cox transformation** with parameter λ is

$$X^{(\lambda)} = \begin{cases} \frac{X^\lambda - 1}{\lambda} & \lambda \neq 0, \\ \log X & \lambda = 0. \end{cases}$$

The family is continuous in λ (the case $\lambda = 0$ is the limit of $(X^\lambda - 1)/\lambda$ as $\lambda \rightarrow 0$, by L'Hôpital's rule). Up to shifts and positive rescaling, familiar cases include $\lambda = 1$ (linear/identity), $\lambda = 1/2$ (square-root), $\lambda = 0$ (logarithm), and $\lambda = -1$ (reciprocal).

The optimal λ can be estimated by maximum likelihood under the assumption that the transformed variable $X^{(\lambda)}$ is normally distributed. Writing $\mathbf{x}^{(\lambda)} = (x_1^{(\lambda)}, \dots, x_n^{(\lambda)})^\top$, the log-likelihood is

$$\ell(\lambda) = -\frac{n}{2} \log \hat{\sigma}^2(\lambda) + (\lambda - 1) \sum_{i=1}^n \log x_i,$$

where $\hat{\sigma}^2(\lambda) = n^{-1} \sum_{i=1}^n (x_i^{(\lambda)} - \bar{x}^{(\lambda)})^2$ is the maximum-likelihood estimate of the transformed variance (using divisor n , not the Bessel-corrected sample variance of Lecture 2). Maximising $\ell(\lambda)$ gives the value $\hat{\lambda}$ that maximises the Gaussian profile likelihood for the transformed data.

Remark 3.3. Tree-based models (decision trees, random forests, gradient boosting machines) are invariant to any monotone increasing transformation of the features, since they partition the feature space by comparing X to thresholds and the ordering of values is preserved under such transformations. Applying a log or Box-Cox transformation to features before fitting a random forest has no effect on predictions. It may, however, still be useful to transform the *target* in a regression tree setting, since the squared-error loss used at the leaves is sensitive to the scale and distribution of the target.

Example 3.4. A dataset of 500 house sale prices (in thousands of pounds) has mean $\bar{x} = 312$, median 286, and standard deviation $s = 198$. The skewness is 1.74, indicating a pronounced right tail. Applying the Box-Cox transformation with $\hat{\lambda} = 0.12$ (close to the log case) reduces the skewness of the transformed variable to 0.09. A linear regression model fitted on the log-transformed prices produces substantially smaller residuals and better-calibrated prediction intervals than one fitted on the raw prices, because the error variance is more nearly constant across the range of the target.

3.3 The Yeo-Johnson extension

The Box-Cox family is defined only for $X > 0$, which rules out many variables of practical interest: temperature anomalies, financial returns, and any feature centred at zero can take negative values. Yeo and Johnson (2000) extend the family to the whole real line.

Definition 3.5. For $X \in \mathbb{R}$, the **Yeo-Johnson transformation** with parameter λ is

$$X^{(\lambda)} = \begin{cases} \frac{(X+1)^\lambda - 1}{\lambda} & \lambda \neq 0, X \geq 0, \\ \log(X+1) & \lambda = 0, X \geq 0, \\ -\frac{(-X+1)^{2-\lambda} - 1}{2-\lambda} & \lambda \neq 2, X < 0, \\ -\log(-X+1) & \lambda = 2, X < 0. \end{cases}$$

Remark 3.6. For $X > 0$ the Yeo-Johnson formula coincides with the Box-Cox transformation applied to $X + 1$. This does not mean the two families are interchangeable in practice, however: fitting Box-Cox to X and fitting Yeo-Johnson to the same X are different optimisation problems (the latter transforms $X + 1$, not X), and the two procedures can select different optimal λ and produce different fitted transformations even when every observed value happens to be positive. The optimal λ is again chosen by maximum likelihood under an assumed normal target distribution, exactly as in Section 3.2. Given its generality, most software implementations (including the one used in the workshop) default to Yeo-Johnson rather than Box-Cox, reserving Box-Cox for settings where positivity is guaranteed by the variable's definition and a simpler, more interpretable form is preferred.

4 Data Leakage and the Train/Test Split

A fundamental requirement of any honest model evaluation is that the test data plays no role in any decision made during training – including decisions about how to transform the data. Violating this requirement is called *data leakage*, and it causes the test performance to be an optimistic overestimate of the true generalisation performance.

Definition 4.1. **Data leakage** occurs when information from outside the training set influences the model or its preprocessing, causing the evaluation to overestimate the model's performance on genuinely unseen data.

Leakage in the transformation stage arises as follows. Suppose the dataset is split into a training set $\mathcal{T}$ and a test set $\mathcal{S}$. If the parameters of a scaler (say, $\bar{x}$ and s for standardisation) are estimated on the full dataset $\mathcal{T} \cup \mathcal{S}$ and then used to transform both sets, the transformation of the training data depends on the test observations. The model is therefore fitted on data whose preprocessing has been influenced by the test set, breaking the independence assumption on which the test evaluation rests.

Proposition 4.2. Let $\bar{x}_{\mathcal{T}}$ and $\bar{x}$ denote the sample means of a variable X on the training set and full dataset respectively. If the test set has size $n_{\mathcal{S}}$ and mean $\bar{x}_{\mathcal{S}}$, then

$$\bar{x} = \frac{n_{\mathcal{T}} \bar{x}_{\mathcal{T}} + n_{\mathcal{S}} \bar{x}_{\mathcal{S}}}{n_{\mathcal{T}} + n_{\mathcal{S}}}.$$

The full-dataset mean $\bar{x}$ therefore encodes the test mean $\bar{x}_{\mathcal{S}}$, and any transformation using $\bar{x}$ carries test information into the training step.

The correct procedure is the following.

- (1) Split the data into training and test sets before any transformation is applied.
- (2) Estimate all transformation parameters (scaler statistics, encoder mappings, Box-Cox or Yeo-Johnson λ , empirical quantiles) on the training set only.
- (3) Apply those parameters to transform both the training and test sets.

In practice, this is enforced by wrapping transformations and the learning algorithm together in a *pipeline*, so that all fitting is performed inside cross-validation folds rather than on the data as a whole. The `sklearn Pipeline` class is designed precisely for this purpose and will be used throughout the workshop sessions.

Remark 4.3. Target encoding is a particularly common source of leakage because the target mean for each category is computed using y values from the training set. If leave-one-out or cross-validation-based target encoding is not used, the encoded value for observation i in the training set will include y_i itself, directly leaking the target into the feature. This is especially damaging for categories with few observations, where the category mean is dominated by the single observation being encoded. Frequency encoding and the hashing trick (Section 2.4) are immune to this particular failure mode, since neither uses y at all, though they remain subject to the ordinary requirement that any category counts be computed on the training set alone.

5 Exercises

Exercise 5.1 (Applied). A training dataset contains a single feature X with values $\{1, 2, 4, 5, 8, 10, 200\}$ and a test dataset with values $\{3, 6, 9\}$.

- (a) Apply min-max scaling using parameters estimated on the training data. What are the scaled values for the training set? What is the scaled value for the test observation $x = 9$? Does $x = 9$ lie in $[0, 1]$ after scaling? Should it?
- (b) Repeat using the robust scaler (median and IQR estimated on the training data, with quartiles computed by linear interpolation as most statistical software does by default). How does the treatment of the outlier 200 differ between the two scalers?
- (c) A colleague argues that since the test values $\{3, 6, 9\}$ are not extreme, it does not matter whether scaling parameters are estimated on the training set or the full dataset in this case. Give a counterargument.

Solution. (a) Training values: $\{1, 2, 4, 5, 8, 10, 200\}$, so $x_{\min} = 1$ and $x_{\max} = 200$. The scaled training values are $(x_i - 1)/199$ for each x_i , giving approximately $\{0, 0.005, 0.015, 0.020, 0.035, 0.045, 1\}$. The test value $x = 9$ maps to $(9 - 1)/199 \approx 0.040$, which lies in $[0, 1]$. A test observation need not lie in $[0, 1]$ after scaling with training parameters: any test value outside $[x_{\min}, x_{\max}]$ of the training data will map outside $[0, 1]$. This is expected and does not indicate an error; the scaler should not be re-fitted on test data to avoid it.

- (b) Training median: 5. Using linear interpolation on the sorted training values $\{1, 2, 4, 5, 8, 10, 200\}$ ($n = 7$), Q_1 falls at position $0.25 \times 6 = 1.5$, interpolating between the 2nd and 3rd order statistics: $Q_1 = 2 + 0.5(4 - 2) = 3$. Similarly Q_3 falls at position $0.75 \times 6 = 4.5$: $Q_3 = 8 + 0.5(10 - 8) = 9$. Hence $\text{IQR} = 9 - 3 = 6$. Robust-scaled training values: $(x_i - 5)/6$, giving

approximately $\{-0.667, -0.5, -0.167, 0, 0.5, 0.833, 32.5\}$. The outlier 200 maps to $(200 - 5)/6 = 32.5$ rather than to 1 as in min-max scaling, so the non-outlying values are not compressed. The test value $x = 9$ maps to $(9 - 5)/6 \approx 0.667$.

- (c) The argument confuses outcome with process. The point of estimating scaling parameters on the training set alone is to ensure that the model evaluation is honest: the test set must be treated as genuinely unseen. Even if the numerical difference between training-only and full-dataset parameters happens to be small in a particular case, using the full dataset sets a precedent that will cause real damage in settings where the test distribution differs from the training distribution, or where the test mean happens to be far from the training mean. The correct procedure must be followed as a matter of principle, not selectively applied based on whether leakage appears to matter in a specific instance.

□

Exercise 5.2 (Mathematical). Let $X_1, \dots, X_n$ be i.i.d. positive random variables with $\mathbb{E}(\log X_i) = \mu$ and $\text{Var}(\log X_i) = \sigma^2$. That is, $\log X_1, \dots, \log X_n$ are i.i.d. with mean μ and variance σ^2 (not necessarily normal, but with these two moments).

- (a) Using the delta method, show that for large n the geometric mean $G = (\prod_{i=1}^n X_i)^{1/n}$ satisfies

$$\sqrt{n}(G - e^\mu) \xrightarrow{d} N(0, \sigma^2 e^{2\mu}).$$

- (b) Let $\bar{Y} = n^{-1} \sum_{i=1}^n \log X_i$. Show that $\bar{Y}$ is a consistent estimator of μ and state its asymptotic distribution.
- (c) Explain, in terms of parts (a) and (b), why G , rather than $\bar{X} = n^{-1} \sum_{i=1}^n X_i$, is the appropriate estimator when the target measure of centre is the median e^μ of a log-normal distribution, particularly when σ is large.

Solution. (a) Let $Y_i = \log X_i$. By the central limit theorem, $\sqrt{n}(\bar{Y} - \mu) \xrightarrow{d} N(0, \sigma^2)$, where $\bar{Y} = n^{-1} \sum_{i=1}^n Y_i$. Since $G = e^{\bar{Y}}$, the delta method applied to $g(y) = e^y$ gives, since $g'(y) = e^y$,

$$\sqrt{n}(g(\bar{Y}) - g(\mu)) \xrightarrow{d} N(0, [g'(\mu)]^2 \sigma^2) = N(0, e^{2\mu} \sigma^2).$$

Since $g(\bar{Y}) = e^{\bar{Y}} = G$, this is exactly the stated result for the geometric mean G .

- (b) $\bar{Y}$ is the sample mean of the i.i.d. sequence $\log X_1, \dots, \log X_n$, each with mean μ . By the law of large numbers, $\bar{Y} \xrightarrow{p} \mu$, so $\bar{Y}$ is consistent. By the central limit theorem, $\sqrt{n}(\bar{Y} - \mu) \xrightarrow{d} N(0, \sigma^2)$.
- (c) To estimate e^μ (the median of X under log-normality), consider the ordinary sample mean $\bar{X}$ and the geometric mean $G = e^{\bar{Y}}$. Part (a) shows that G is consistent for e^μ , with asymptotic variance $\sigma^2 e^{2\mu}/n$. Under the additional assumption that X is exactly log-normal, $\bar{X}$ does not estimate e^μ : rather, $\bar{X} \xrightarrow{p} \mathbb{E}(X) = e^{\mu + \sigma^2/2}$, whereas $G \xrightarrow{p} e^\mu$. Working via G therefore targets the median e^μ directly, whereas $\bar{X}$ targets the larger quantity $e^{\mu + \sigma^2/2}$. When σ is large the gap between these two quantities is substantial, and $\bar{X}$ becomes a poor representative of the distribution's median, even though it remains a perfectly good estimator of $\mathbb{E}(X)$ itself.

□

Further Reading

Box and Cox (1964) give the original treatment of the power transformation family introduced in Section 3.2, including the maximum-likelihood approach to selecting λ used throughout that section. Yeo and Johnson (2000) extend the family to variables taking zero or negative values, as discussed in Section 3.3. Lindholm et al. (2022) treat scaling, encoding, and pipeline construction as part of a unified machine learning workflow, in the spirit of Section 4; James et al. (2023) offer a complementary applied treatment of feature scaling and the regularisation contexts in which it is most consequential.

References

- Box, G.E. and Cox, D.R., 1964. An analysis of transformations. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 26(2), pp.211–243.
- James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J., 2023. *An introduction to statistical learning: with applications in Python*. New York: Springer. Freely available at <https://www.statlearning.com>.
- Lindholm, A., Wahlström, N., Lindsten, F. and Schön, T.B., 2022. *Machine learning: a first course for engineers and scientists*. Cambridge University Press.
- Yeo, I.K. and Johnson, R.A., 2000. A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4), pp.954–959.